Server

- Storage
  - Create Partition
  - LVM (Logical Volume Mgmt)
  - Disk Usage
- File System
  - Link
    - Linked Directories
    - Create a link
      - Inode
  - Mount
    - /etc/fstab Config Mount
- Stratis
- SSH, SCP: remote acc
  - SSH Agent & Key Forwarding
  - SCP: inter-machines file copy
- OS ISO Image
  - Setup on VMware vSphere
  - Bootloader
    - GRUB & GRUB2
      - GRUB
      - GRUB2
  - Runlevel: 0-6
- HW Check
- Kernel & Service
  - Kernel
  - Service
  - Environment Variables
- Package Mgmt
- Network
  - Test Connection
  - Network Interface Config
  - Bond
  - DNS
  - nmcli Network Manager
  - Firewall
- Root / Superuser/ Sudo
  - User, Group
    - Special Permissions
- ACL
- Read files
- Write files (ansible, etc.)
  - HereDoc
- Find files
- Process
  - Process vs Job:
- Utils
- Sed
- AWK
- VIM
- Job Scheduler
- Monitor

- Datetime
  - chrony
- Archive
- FTP
- SELinux
- Fine Tune Sys Performance
- Network FS
  - NFS
  - Samba
  - NAS: Network Attached Storage
  - SAN: Storage Area Network
- Podman

Storage

`/dev/sda1` Device SCSI <**device#**(a,b,c)> <**partition#**(1,2,3)>

Mount Points:

- `/` root, for OS per se
- `/customApp` docker, etc
- `swap` usually 2x of mem, or min. 2Gib

Raw: no disk partition, harder partition in the future, e.g. EC2

Create Partition

```
fdisk /dev/sdb
n # add new partition
# partition size = 1e - last sector
w # write
```

LVM (Logical Volume Mgmt)

Disk λ-φ mapping, only works after OS installation.

PV: Physical Volume

VG: Volume Group, a set of PV.

LV: Logical Volume

```
pvcreate <PV1> <PV2>
vgcreate <vg_name> <PV1> <PV2>
lvcreate -L <size> -n <lv_name> <vg>

lvdisplay

mkfs.ext4 <lv_path>

lvextend -L <size_to_ext (+10G)> <lv_path>
```

Disk Usage

```
df
du / -h | sort -nr | less # -nr: numeric, reverse sort
```

File System

`tree` show folder structure

`cd /` to root dir

`~` home of current user, e.g. `/home/ubuntu`

Extended FS: `ext4`, `xfs`

```
mkfs.ext4 /dev/sdc2
```

Link

Linked Directories

```
/usr/tmp -> /var/tmp
/bin -> /usr/bin
/lib -> /usr/lib
```

N.B. `/var/tmp` is diff. from `/tmp`, files in `/tmp` will disappear after reboot.

Create a link

`ln <target_filepath> <link_name>`

Link: @ `~/Desktop/`; Actual file @ `/var/tmp/`

Write to link will also write to orig. file

```
cd /var/tmp/
touch file.txt

cd ~/Desktop/
ln -s /var/tmp/file.txt # -s: soft link
ls -l
```

- Hard Link: only works w/in same partition
- Soft (Symbolic) Link
- Hard vs. Soft: Soft points to orig. file, Hard points to `inode` in the hard disk, i.e. Hard & Orig. points to the same `inode`.
- *Soft* → Orig. → Inode; *Hard* → Inode

Inode

- var: `nlink` for link count. Inode will be deleted if `nlink` = 0 and file is not opened, i.e. not in RAM.
- Check: `ls -l <filename>` or `stat <filename>`

Mount

`mount -t <FS_type (ext4, xfs)> <partition> <mount_point>`

`/etc/fstab` **Config Mount**

```
# <file system> <mount point> <type> <options> <dump> <pass>
  /dev/sdc2     /home/app     ext4   defaults  0      2
```

`<pass>` check FS `fsck`, `1`: root, `2`: others, `0`: no check

`mount -a`

`fsck` check FS, i.e. partition, do NOT do this to mounted partition

Stratis

```
# Create hard disks on VM manager

# Create pool
stratis pool create <pool> <device1> <device2>
stratis pool list

stratis pool add-data <pool> <device>

# Create FS
stratis filesystem create <pool> <fs>
stratis filesystem list

mkdir <dir>
mount /dev/stratis/<pool>/<fs> <dir>
lsblk

# Create SNAPSHOT
stratis filesystem snapshot <pool> <fs> <snapshot_filename>

# Add entry to /etc/fstab
--- /etc/fstab
UUID=<UUID> <dir> <fs_type> defaults,x-systemd.requires=stratisd.servic
```

`SSH, SCP` : remote acc

`ssh <username>@<ip>`

Remote acc. w/ SSH key

```
# Run on Master
ssh-keygen
ssh-copy-id <slave_username>@<slave_ip_adr>
ssh <slave_username>@<slave_ip_adr>
```

Troubleshoot

```
# Login to Slave
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
```

Master: `192.168.119.217` Slave: `192.168.119.226`

Bastion / Jump Host: host created by client, offered to us to run scripts, ansible, etc.

SSH Agent & Key Forwarding

```
eval $(ssh-agent) # eval exec arg qua shell cmd.
ssh-add ~/.ssh/id_rsa

ssh -A <username>@<jump-server> # -A: key forward
```

`/etc/ssh/sshd_config`

```
# Config idle timeout
ClientAliveInterval 600 # 10min
ClientAliveCountMax 0

# Disable Root login
PermietRootLogin no

# Disable empty password
PermitEmptyPasswords no

# Limit user SSH acc
AllowUsers user1 user2

```

```
# Use different port
Port <port#>

# Locale Setup
AcceptEnv LANG LC_CTYPE LC_NUMERIC LC_TIME LC_COLLATE LC_MONETARY LC_ME
AcceptEnv LC_PAPER LC_NAME LC_ADDRESS LC_TELEPHONE LC_MEASUREMENT
AcceptEnv LC_IDENTIFICATION LC_ALL LANGUAGE
AcceptEnv XMODIFIERS
```

`SCP` : inter-machines file copy

```
scp <src> <dest>

# Local -> Remote
scp <filename> <username>@<ip>:<path>

# Remote -> Local
<username>@<ip>:<path> .
```

OS ISO Image

Setup on VMware vSphere

Action → Edit settings

- Hard Disk: Disk Provisioning: *Thin Provision*
- Network: *VM network*
- CD/DVD drive: Datastore ISO file; connected checkbox
  - Path: Datastore → SynologyISO → ISO → choose OS

Bootloader

BIOS/UEFI → GRUB(2) → `initrd` → OS kernel (ver) → Modules

GRUB & GRUB2

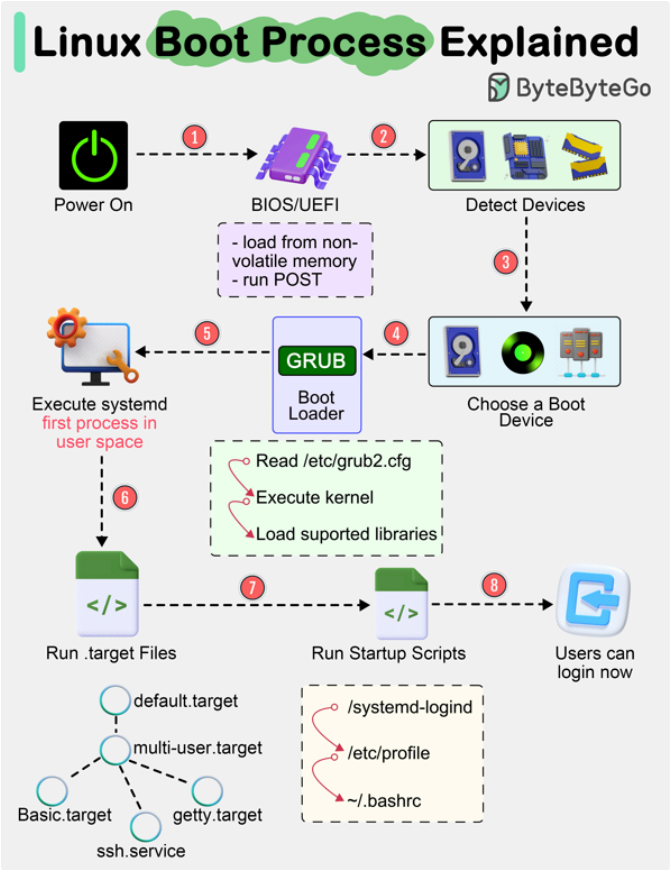
GRUB

- legacy
- `menu.lst`, `grub.conf`
- difficult to modify

GRUB2

- `/etc/default/grub` config here
- `/boot/grub/grub.cfg` read by bootloader, do not modify this file, `/boot/grub2/grub.cfg` for RHEL
- Hidden boot menu (press `Shift`)

Run kernel & initrd



Path: `/boot`

`modprobe`: cmd to decide which kernel modules to load / unload, depends on `insmod`. check `/lib/modules/<module_name>/kernel`

Runlevel: 0-6

- Referred to as targets.
- `systemctl get-default` or `who -r`

HW Check

```
lscpu
lsblk # disks, partitions
lspci
```

Kernel & Service

Kernel

```
dmesg | less # Kernel boot log
dmesg | grep -i "pattern" # -i: ignore case sensitivity

modprobe <module> # load kernel module
```

Service

```
systemctl status <service_name>

systemctl enable/disable <service_name> # auto start service on boot
systemctl start/stop <service_name>
```

```

systemctl daemon-reload # reload systemd config
```

`<service-name>.service`

```
WorkingDirectory=/home/<username>/<proj>
# dir abs path
```

Environment Variables

`/etc/environment` env var.

`env` get all env var

Write Env var

```
export KUBECONFIG="path/to/yml"
echo $KUBECONFIG
unset KUBECONFIG

# Write permanently
vi ~/.zshrc
--- .zshrc
export KUBECONFIG="/path/to/yml"

source ~/.zshrc
```

All services graceful shutdown before reboot: `shutdown -r now`

Package Mgmt

```
apt update && apt upgrade -y
# refresh local pkg idx && upgrade installed pkg
```

RHEL, CentOS

```
rpm -qa ( | grep <package>) # list package

dnf update -y
dnf remove <package> -y
```

Repos

```
cd /etc/yum.repos.d/
sudo vi redhat.repo
--- redhat.repo
[BaseOS]
name=BaseOS Repo
baseurl=http://content.example.com/rhel9/BaseOS
enabled=1
gpgcheck=0

dnf repolist
```

Network

Test Connection

```
ping <ip>
nc -zv <ip> <port>
```

```
timeout 2 echo >/dev/tcp/127.0.0.1/9090 && echo "successful" || echo "fa
```

Network Interface Config

Path: `/etc/netplan/`

`00-installer-config.yaml` or `50-cloud-init.yaml`

```
ip a # show IP

# ss: Socket Statistics, replacement of netstat
ss -tnlp # TCP, Numerical IP, disp Listening skts, show PID.
```

Bond

Path: `/etc/netplan`

Multi NICs (Slave) exist, set mode to increase BW.

Modes

0. balance-rr (Round-Robin)
1. active-backup: 1 active, if it fails, another one will takeover.
2. balancer-xor: connexion based on hash policy, usually replaced by mode 4.
3. broadcast
4. 802.3ad: dynamic link aggregation, if switch supports link aggregation.
5. balance-tlb (Transmit Load Balancing): outbound traffic based on current load, choose the least busy port. No special switch required.
6. balance-alb (Adaptive LB): in & out bound traffic. No special switch required.

Scenario	Mode
Simple setup, connect 2 servers directly.	`0`
Switch qui support Link Aggregation	`4`
Dumb switch	`6`

`netplan apply` after mod netplan

DNS

`/etc/hosts`

- Mapping IP & DNS
- Pre-DNS lookup
- Higher priority, for DNS blocking. Rules def @ `/etc/nsswitch.conf`

`nslookup` name server lookup, query the DNS

`ethtool <intf>`

Network Files & Dir: `/etc/resolv.conf`, `/etc/hosts`, `/etc/hostname`, `/etc/nsswitch.conf`

`nmcli` Network Manager

```
hostnamectl set-hostname <server1.example.com>

nmcli c show # c: connection
sudo nmcli c mod <intf ("eth0", "ens192")> ipv4.method manual \
ipv4.addr 172.25.250.100/24 \
ipv4.gateway 172.25.250.1 \
ipv4.dns 172.25.250.1

nmcli c mod <intf> +ipv4.addr <IP CIDR> # add IP to intf

nmcli c up <intf>

ip a show
ip route
cat /etc/resolv.conf # DNS
hostname
```

IPv4 & Ipv6 adr config stored @ `/etc/NetworkManager/system-connections`, (`/etc/sysconfig/network-scripts/ifcfg-<intf>` deprecated)l

Firewall

```
firewall-cmd --list-all

firewall-cmd --list-services
firewall-cmd --permanent --add-service=<service>
firewall-cmd --permanent --add-port=<port#>/<ptcl>

firewall-cmd --get-active-zones/--get-default-zone
firewall-cmd --info-zone=<zone>

firewall-cmd --get-policies
firewall-cmd --info-policy=<policy>

firewall-cmd --add-rich-rule='rule family="ipv4" source address="<ip>"
firewall-cmd --reload
```

`masquerade`: in zone & policy info, i.e. NAT

Root / Superuser/ Sudo

Enter root / superuser

```
su -
su <username> # act as a spec. user
sudo su -
```

Modify `sudoers` list

```
su -
sudo visudo
<username> ALL=(ALL:ALL) ALL

# Alt
usermod -aG wheel <username> # Group: wheel, sudo perm group
```

`ALL=(ALL:ALL) ALL`: 1e: allow @ any machine, `(ALL:ALL)` as any user and as any group, 2e: run all cmd.

User, Group

```
useradd <username> -s <login_shell (/bin/bash, /sbin/nologin)>
usermod -aG <group1,group2...> <username>
userdel -r <username>

# Add users to group
gpasswd -M <user1>,<user2> <group>

```

```
# Remove user from group
gpasswd -d <user> <group>

chage -m <passwd_change_min_intv> -M <passwd_change_max_window> \
  -E <exp_date> <username>
```

`-a` : append, `-G` : secondary groups. only using `-G` will overwrite user original groups.

> Check available shell: `/etc/shells` , `sbin/nologin` ,
>
> Check current shell: `echo $0`

Special Permissions

SUID (4): always exec file as Owner User, regardless of the users. Once cmd starts, sys treats proc as if Owner User is running it, pro tem.

`-rwsr--r--`

```
chmod u+s <filename>
```

SGID (2):

- File: being exec as group owner.
- Dir: all files w/in have group ownership of that dir group owner. *For dir qui are used in collab b/w members of a group.*

`-rwxrws---`

```
chmod g+s <filename/dir>
# Alt
chmod 2770 <filename/dir>
```

Sticky bit (1): only owner & root of a file can delete the file.

```
chown -R <username>:<group> file1 file2
```

Show `uid` , `gid` , `groups`

```
id <username>
```

`/etc/passwd` & `/etc/shadow` : `shadow` file encrypts the password

`/etc/login.defs` : def `useradd` policy

List group members

```
getent group <group_name>
```

ACL

```
setfacl -m d:u:<username>:<perm> <filepath>
setfacl -Rm g:<group>:<perm> <dir>
-m: modify
-x: remove
```

```
-R: recursive
d: default, new files will inherit ownership attr.

getfacl <filepath>
```

Read files

```
head -n <int> <filename>
tail -n <int> <filename>
```

Scroll thru text & Search

```
less <filename>
  # In less:
  /<keyword> # search keywords in the texts.
```

Write files (ansible, etc.)

```
echo "hi" > file.txt  # overwrite
echo "hi" >> file.txt # append
> <filename> # clear file content

echo "hi" | tee file.txt
echo "hi" | tee -a file.txt # append
```

HereDoc

```
tee dev_readonly.sql <<-EOF
  CREATE ROLE "{{name}}" WITH LOGIN PASSWORD '{{password}}' VALID UNTIL
  GRANT ro TO "{{name}}";
EOF
```

`tee` read stdin, write it to stdout & file.

`-` ignore any leading tabs.

`EOF`: delimiter tk, arbitrary, could be anything. When delimiter tk has quotation marks: `"EOF"`, it disables variable expansion, i.e. print everything literally, e.g.

```
USER="alice"
cat <<EOF
Hello $USER
The time is $(date +%H:%M)
EOF

Output:
Hello alice
The time is 15:30

cat <<"EOF"
Hello $USER
The time is $(date +%H:%M)
EOF

Output:
Hello $USER
The time is $(date +%H:%M)
```

`<dir>` usually `/home`, `/usr`

`xargs`: like grep, read items from stdin, and build & exec cmds. e.g. `pgrep -f vault | xargs kill`

Find files

```
find [path] [options] [expr]

sudo find / -name *<filename>* 2> /dev/null

# Find files 10M < size < 100M, GUID set
find <dir> -size +10M -size -100M -perm -2000
```

`**` double-stars qua wildcards to find pattern.

`2` File Descriptor for `stderr`, `2> /dev/null` redirect `stderr` to black hole.

N.B. `2>` w/ No Space in-between.

`-perm`

- `mode`: match exact mode.
- `-mode`: at least the spec. bits are set, AND logic, e.g. 2755 will match -2000
- `/mode`: any of the set bit, OR logic.

Process

```
ps aux
kill -9 <pid> # -9: force kill
```

`a` show proc for all users

`u` disc proc user/owner

`x` show proc not attached to a terminal.

Process vs Job:

Process: any running program w/ own address space.

Job: concept used by shell. Any program you interactively start that doesn't detach, i.e. not a daemon.

Utils

```
echo -ne "$i \t"
# -n: do not output the trailing newline
# -e: enable interpretation of backslash escapes

h=$(hostname)

wc -l # word count #lines

echo $0 # check current shell
echo "" # interpret var: $, cmd sub: ``, backslash: \
echo '' # everything taken literally
echo `` # taken as command
```

Sed

Remove lines w/ `##` even w/ whitespaces before it

```
sed -i '/^[[:space:]]*##/d' file.txt
```

AWK

Text processing

`NR` #records, rows

`NF` #fields, columns

```
awk [opt] 'pattern {action}' input-file > output-file
-F: custom field separator, default: space
-f: read awk program file
-v: assign var before exec
```

```
# Unseal Vault
keys=$(awk '{print $NF}' keys.txt)
for key in $keys; do
    echo "--- Unsealing w/ key: $key ---"
    vault operator unseal "$key"

    if vault status | grep -q "Sealed.*false"; then
        echo "✅ Vault is unsealed!"
        break
    fi
done

# Delete multi PVC
kubectl -n <ns> get pvc | while read pvc
do
    kubectl -n <ns> delete pvc `echo $pvc | awk '{print $1}'| grep -v N
done

# Parse root_tk.txt to get root token
export VAULT_TOKEN="$(awk '{print $4}' root_tk.txt)"
```

```
# Replace string
awk '{ gsub(/gianna/, "Giovanni"); print}' subst.txt > subst1.txt

# In-place replace edit
awk '{ gsub(/gianna/, "Giovanni"); print }' file.txt > file.txt.tmp \
&& mv file.txt file.txt.bak \
&& mv file.txt.tmp file.txt
```

VIM

```
vi file.txt
# Cmd mode:
:wq -> Enter # Save and Exit
gg           # 1e line of doc
G            # Last line of doc
dd           # delete line

i            # enter insert mode
/<keyword>   # search keyword next / prev by hitting n / Shift+n
```

Job Scheduler

`crontab -e` open and mod cron schedule file as a user.

```
crontab -e
* * 29 2 * echo "This cron only runs every min, on 29 Feb"
# Cron syntax: <min> <hr> <dayOfMonth> <month> <dayOfWeek[0-7]>
```

`at` one-off job scheduler

```
at <time>
commmands...
Ctrl+D

atq       # current queued job
atrm <id> # remove job
```

Monitor

`uptime` uptime & load average in 1, 5, 15 min, often cf. #cores.

Waiting tasks = load_avg - (#cores * util_CPU), `uptime` - (`nproc` * `top`).

`top` real-time proc. monitor

`free` check free mem & swap space.

`tcpdump` network packet analyser, ~wireshark.

`/var/log/`

Limit sys log size:

```
--- journald.conf
SystemMaxUse=100M
```

Datetime

`date`

`timedatectl`

`chrony`

`/etc/chrony.conf`

```
server pool.ntp.org iburst
```

```
systemctl restart chronyd
systemctl enable chronyd
```

`chronyc` chrony CLI

Archive

```
tar --gzip -cf /tmp/backup.tar.gz /etc/sysconfig
# -c: create, -x: extract, -f: spec filename (must be last option), -j:
```

FTP

Server

```
vi /etc/vsftpd/vsftpd.conf
--- vsftpd.conf
anonymous_enable=NO
ascii_upload_enable=YES
asci_download_enable=YES
use_localtime=YES


systemctl start vsftpd
systemctl stop firewalld
```

Client

```
ftp <ftp_server_ip>
# Enter server username & password
bin
```

```
hash
put <filename>
# Check server user's home dir
```

SELinux

`sestatus` check SELinux status

Label: `user:role:type:level`

```
ls -lZ
ls -dZ
```

Config

```
/etc/selinux/config
touch /.autorelabel
```

```
# Boolean: run-time switch to enable/disable SELinux policy
getsebool -a
setsebool -P <boolean_name> on

# Change type label
chcon -t <type> <filename (e.g. "/var/www/custom(/.*)?">
semanage fcontext -a -t <type> <filename>
```

Troubleshoot for SELinux denial

```
ausearch -m avc # avc: Access Vector Cache
```

Fine Tune Sys Performance

`tuned`

```
tuned-adm active
tuned-adm list
tuned-adm profile <profile> # change to desired profile
tuned-adm recommend
tuned-adm off
```

`nice` & `renice` Prioritise process

Check sys priority: `top`

`PR` process actual priority: RT-39 (RT, -99, -98,...; HI-LO)

`NI` user space process priority: -20-19 (HI-LO), mapping 0-39 of `PR`, `PR` = `NI` + 20

```
ps axo pid,comm,nice,cls --sort-nice

# start command w/ proc prior
nice -n <prior#> <pname>

# change proc prior (running)
renice -n <prior#> <pid>
```

Network FS

NFS

- Linux to Linux
- Matching UID/GID

Server side

```
dnf install nfs-utils -y
```

```

systemctl enable --now nfs-server

mkdir <server_dir>
chmod -R 777 <server_dir>

--- /etc/exports
<server_dir> <client_ip>(rw,sync,no_root_squash)
# Root squash: map client root user (UID 0) to a less privileged user

exportfs -rv

firewall-cmd --add-service=nfs --permanent
firewall-cmd --reload
```

Client side

```
dnf install nfs-utils -y

mkdir <client_dir>

--- /etc/fstab
<server_ip>:<server_dir> <client_dir> nfs _netdev,vers=4.2,rw 0 0

sudo systemctl daemon-reload
sudo mount -a
```

Alt: `autofs`

```
dnf install autofs -y

vi /etc/auto.master
    /mnt/test /etc/auto.nfs

vi /etc/auto.nfs
    <subdir> -rw,sync  <server_ip>:/nfs/share # /mnt/test/<subdir>

systemctl restart autofs
systemctl enable autofs
systemctl status autofs

cd /mnt/test/<subdir>
```

Samba

- Windows or mix environments
- Rely on user, pasword, AD
- Label path on SELinux

Server side

```
dnf install samba policycoreutils-python-utils -y

--- /etc/samba/smb.conf
[sambashare]
  path = <server_dir>
  browsable = yes
  guest ok = yes
  read only = no


testparm -s # check & verify smb.conf
semanage fcontext -a -t samba_share_t "<server_dir>(/.*)?"
restorecon -Rv <server_dir>

firewall-cmd --add-service=samba --permanent
firewall-cmd --reload

systemctl enable --now smb
```

Client side

```
dnf install samba-client cifs-utils -y

```

```
smbclient -L //<server_ip>/<server_dir> -N

get <filename>

mount -t cifs //<server_ip>/<server_dir> <client_dir> -o guest

--- /etc/fstab
//<server_ip>/<server_dir> <client_dir> cifs _netdev,guest 0 0


mount -a
```

NAS: Network Attached Storage

- a device as a node spec for storage.
- File-level acc, e.g. data requires protection.
  - cf. Block-level and Object-level: unstructured data, large dataset
- Employ NFS or SMB
- TCP/IP ethernet Packets: latency

SAN: Storage Area Network

- Conn. storage devices, separated from LAN
- Block-level acc: seq. of bytes having whole #records, e.g. DB, RAID
- Fibre channel

Podman

```
podman info # check env, registry/repo info
```